PAC-Private Databases

Mayuri Sridhar*
MIT CSAIL
mayuri@mit.edu

Michael A. Noguera*
UIUC
noguera2@illinois.edu

Chaitanyasuma Jain
UW-Madison
cjain5@wisc.edu

Kevin Kristensen
UW-Madison
kckristensen@wisc.edu

Srinivas Devadas
MIT CSAIL
devadas@mit.edu

Hanshen Xiao
Purdue University
hsxiao@purdue.edu

Xiangyao Yu
UW-Madison
yxy@cs.wisc.edu

Abstract

As data collection and sharing becomes more prevalent, quantifying leakage about released data is an increasingly crucial privacy issue. Prior work in private database analytics demonstrates how to provide strong theoretical privacy guarantees through differential privacy (DP). However, these techniques are often limited to specific queries; to the best of our knowledge, among the 19 queries in the TPC-H benchmark which do not directly leak customer information, prior work in DP can handle at most 9 queries, without additional analyst effort.

In this work, we apply the recently-proposed Probably Approximately Correct (PAC) Privacy mechanism in order to provide a *black-box* technique to privatize general SQL queries against membership inference attacks. Naïvely applying PAC Privacy would allow us to privatize any *individual* query. However, databases are an interactive process: a user queries the database, *views the response*, and then chooses their next query. Prior work in PAC Privacy cannot provide any theoretical guarantees in this setting; instead, users would be required to provide all the queries *a priori*, which is a fundamental usability limitation. We construct the first algorithm to allow users to query the database adaptively and prove that algorithmic modifications via independent randomness provide automatic privatization guarantees.

Our privatization layer, PAC-DB, does not require *any* human analysis in order to privately return a response for a general SQL query. PAC-DB is compatible with any database management system and does *not* require a trusted data curator. We provide an open-source implementation, where we privatize all of the 19 queries in consideration from the TPC-H benchmark with customers as our privacy concern. We provide both relative errors and initial performance estimates.

Keywords

PAC Privacy, private databases, membership inference attacks

1 Introduction

Sensitive data is collected and processed at increasingly high rates. If the results of the processing are made public, a natural question to ask is: what is the exposure of the sensitive data given the release of the results?

In many scenarios, either within an organization, or across multiple organizations, large-scale datasets are stored in a database that analysts query for understanding aggregate statistics. Classic applications include healthcare [19] and training large models like GPT [25]. While *any* query on private data will leak *some* information about the input dataset, quantifying and minimizing the leakage has become an increasingly urgent problem in data privacy.

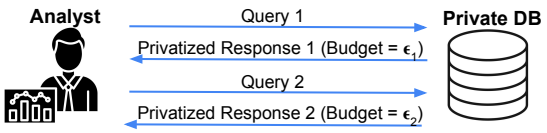
There are several broad approaches to this problem: adding noise to individual query outputs [12, 13, 15], releasing synthetic data representative of the database with quantitatively bounded leakage [33], or encrypting the full dataset [10]. The first two approaches require tightly computing the leakage of a sensitive dataset. The third approach focuses on a different threat model, where the analysts are assumed to be trusted. For the first two approaches, Differential Privacy (DP) [7] has a long history of providing strong theoretical guarantees for individual datapoint privacy through input-independent indistinguishability. Several works [12, 13, 15, 30, 34] demonstrate how to instantiate differential privacy for a variety of SQL queries. However, these techniques require significant trust (e.g., a trusted data curator) and manual analysis of a white-boxed query to determine DP sensitivity. While the trust requirements can be mitigated via cryptographic techniques [23], or by split trust assumptions [1], the white-boxing requirement is harder to fix (cf. Section 8).

In contrast, recent work [31] on Probably Approximately Correct (PAC) Privacy focuses on providing provable privacy guarantees in a *black-box* manner. In particular, for arbitrary mechanisms $\mathcal{M}$, PAC Privacy provides techniques to automatically compute the noise required to privatize $\mathcal{M}$ for a given privacy budget. The key advantage of PAC Privacy is that the noise required to privatize *any* black-box query can be measured *without human analysis*. Practically, PAC Privacy guarantees trade off additional computational power for human analysis.

Prior work in PAC Privacy [27, 31] focuses on the setting where the user receives a **single** privatized release (e.g., the centroids in K-Means). This is fundamentally different from the database setting which is an *interactive process* — that is, the user sends multiple queries, which are often correlated and may be adaptively malicious. This is a harder setting than prior work where an algorithm operates on sensitive data and the privatized output is released *once*.

For practical usage in databases, we consider a model where users query the database with a sequence of queries $Q_1 \dots Q_k$ that are allowed to be *adaptive*. That is, the choice of Q_k may depend on the outputs received from $Q_1 \dots Q_{k-1}$ and be *adversarially* designed to extract secret information about the underlying database — see Figure 1. Naïvely applying prior techniques in PAC Privacy (e.g., [27]) would require *all k queries to be submitted at once*, which is a fundamental limitation to the *usability* of the privatized database. On the other hand, enabling differentially-private databases requires tightly computing sensitivity for each query, which may be

*Equal contribution



Prior Work in PAC Privacy:

Query	Privacy Bound
#1	ϵ_1
#2	ϵ_2
Overall Protocol	Unbounded

This Paper:

Query	Privacy Bound
#1	ϵ_1
#2	ϵ_2
Overall Protocol	$\epsilon_1 + \epsilon_2$

Figure 1: To enable a usable private database, analysts must be able to submit queries, *observe the privatized responses*, and then submit new queries. This is a particularly difficult adversarial model, since the *response* to Query #1 can be used to construct Query #2 in order to leak secret information. In this work, we show how to enable linear privacy bounds across *adaptive, adversarially generated* queries.

arbitrarily complex; this is known to be NP-hard [32]. In this work, we provide provable privacy guarantees for arbitrary *sequences* of general SQL queries.

Our key contributions are as follows:

(1) We provide the first algorithm and theoretical privacy guarantees, for adversarially-generated queries in the PAC Privacy framework. We prove that independent randomness can be exploited in order to automatically measure the noise required for a black-box *sequence* of queries; prior work in PAC Privacy only provides guarantees for a *single* release.
(2) We construct a simple three-step privatization layer which can be built on top of any database in order to privatize SQL queries. This system requires *no* additional trust or human effort for novel queries that have not been seen previously.
(3) We are the first to implement privatization for general SQL queries via PAC Privacy; we provide a complete open-source implementation using Polars and DuckDB.
(4) Using customers as the unit of privacy, we privatize 19 queries in the TPC-H benchmark — the complete set which do not directly leak customer information. We provide detailed results on representative queries, discussing the privacy-utility tradeoffs. We additionally provide performance numbers for our proof-of-concept prototype.

2 Background

In this section, we provide an overview of techniques to provide provable database privacy for general queries.

2.1 Differential Privacy

Differential privacy (DP) has been the standard for providing *provable* privacy guarantees for data analytics. DP allows databases to release aggregate statistics while protecting the membership of individual datapoints, as formally defined in Definition 1.

Definition 1 ($(\epsilon, \bar{\delta})$ Differential Privacy [7]). *Given a data universe $\mathcal{X}^*$, we say that two datasets $\mathcal{S}, \mathcal{S}' \subseteq \mathcal{X}^*$ are adjacent, denoted as $\mathcal{S} \sim \mathcal{S}'$, if $\mathcal{S} = \mathcal{S}' \cup s$ or $\mathcal{S}' = \mathcal{S} \cup s$ for some additional datapoint s. A query Q is said to be $(\epsilon, \bar{\delta})$-differentially-private (DP) if for any pair of adjacent datasets $\mathcal{S}, \mathcal{S}'$ and any set o in the output space O of Q, it holds that* $\Pr(Q(\mathcal{S}) \in o) \leq e^{\epsilon} \cdot \Pr(Q(\mathcal{S}') \in o) + \bar{\delta}$.

While DP provides strong theoretical guarantees, it typically requires query *white-boxing* in order to support varying aggregations. That is, standard DP techniques require knowing the sensitivity of the query in order to compute the required noise for privacy. Here, the sensitivity of a query Q is defined as the maximum change in Q due to a single individual's data [7]. For simple queries such as "count," the sensitivity is known to be 1; however, even for a simple count query with joins, the sensitivity is no longer easy to define. Queries with arbitrary aggregation functions or complex joins require significant human input to provide meaningful privacy guarantees via computing tight sensitivity bounds.

Early work in differentially-private databases, like Privacy Integrated Queries (PINQ), Weighted Privacy Integrated Queries (wPINQ) and GUPT [17, 20, 24], focused on building specialized query platforms to enable private databases. Due to the lack of large-scale adoption of these specialized systems, [13] designed FLEX to be a simple layer on top of any database management system (DBMS), which allows queries to be privatized. [13] defines *elastic sensitivity* to allow differentially-private count queries with joins. That is, a join may create an arbitrary number of duplicates corresponding to a single datapoint; in this case, the sensitivity scales with the maximum number of duplicates. Elastic sensitivity computes an upper bound on the sensitivity of a join by measuring the maximum frequency of varying attributes in the database. We note that this is an upper bound for *local* sensitivity, which depends on the provided ("true") database. Recent work [5] discusses how to adaptively estimate sensitivity in the streaming setting; however, this is hard to generalize for more complex queries. A key limitation of both [13] and [5] is that only "count" queries are supported.

A follow-up work, CHORUS [12], expands the set of queries significantly, while still providing strong DP guarantees. In particular, CHORUS provides a *query rewriting* framework which allows for more complex queries like "sum" or "average" aggregations to have bounded sensitivity. That is, a sum query would be rewritten with lower and upper bounds for individual data elements. This rewritten query can then be analyzed for sensitivity and a differentially-private result is returned.

While CHORUS can technically be used to enable "MAX" queries, it requires significant analyst effort. In particular, the query must be specified along with a "scoring function", which enables the use of the exponential mechanism [8]. Generally, more complex relations which can be supported by techniques like the shifted inverse mechanism [9] are not supported by either FLEX or CHORUS without significant analyst effort (i.e., writing appropriate mechanisms for bounding the sensitivity of *each* individual aggregation function). In Table 1, we break down 19 out of the 22 queries in the TPC-H benchmark and discuss which queries can be privatized without significant analyst effort. These 19 queries represent all the queries in the benchmark which do not directly leak customer data; in our

Table 1: We break down 19 out of the 22 queries in the TPC-H benchmark [28] into their aggregation types; we then identify techniques which can privatize them without significant analyst effort. Several queries involve *multiple* aggregation types (e.g., Q1 uses COUNT as well as SUM and AVG). In these cases, we identify all the categories they require; that is, FLEX would only partially answer Q1's requirements. There are several queries that are within the "Other" category. The simplest query in this category is a sum of "discounted prices" (e.g., Q3) — rather than being a direct sum, this becomes a sum of a *product*, requiring computing and multiplying individual sensitivity computations. We note that while Q11 cannot technically be privatized by these techniques, it has *no* dependence on customers; thus, it can be run unmodified.

	Technique		
Aggregation Type	FLEX [13]	CHORUS [12]	PAC-DB
COUNT [Q1, Q4, Q13, Q16, Q21, Q22]	✓	✓	✓
SUM or AVG [Q1, Q12, Q17, Q22]	×	✓	✓
MIN or MAX [Q2, Q15]	×	×	✓
Other Functions [Q1, Q5, Q6, Q7, Q8, Q9, Q11*, Q14, Q15, Q19, Q20]	×	×	✓

work, we privatized *all 19 queries* without any changes required from the analyst.

Further, while prior works provide strong theoretical guarantees, they rely on the existence of a *trusted data curator*. That is, the database is managed by an honest party to provide meaningful privacy guarantees; this honest party is typically required for DP. In FLEX, the maximum frequency must be computed by an honest party that has access to the provided or true database values. In CHORUS, the query rewriting framework requires the database management system (DBMS) to clip the queries to provide bounded sensitivity; the choice of these values will significantly affect the utility of the result. This is especially true for queries like Q21 (a count with an antijoin) where sensitivity can grow rapidly. Further, this is not a one-time cost; that is, if the underlying data is updated, the optimal choice of these hyperparameters changes as well.

2.2 PAC Privacy

Recent work [31] on Probably Approximately Correct (PAC) Privacy focuses on providing provable privacy guarantees in a *black-box* manner. In particular, for arbitrary mechanisms $\mathcal{M}$, PAC Privacy provides techniques to automatically compute the noise required to privatize $\mathcal{M}$ for a given privacy budget.

The key advantage of PAC Privacy is that the noise required to privatize *any* black-box query can be measured without human input. While the computation of sensitivity for DP typically requires analyzing the distribution of the input data and the query itself, PAC Privacy simply requires repeated simulations. As modern databases easily handle complicated queries and take advantage of ever-increasing computational power, we posit that PAC Privacy is compelling for the database domain. However, we note that PAC Privacy operates under a different threat model, and thus provides semantically different privacy guarantees, as we discuss more in Section 3.2.3. Prior work in PAC Privacy [27, 31] focuses on the setting where the user receives a **single** privatized release. However, in the online analytical processing (OLAP) database setting [3, 18], queries are more interactive and explorative, and later queries can be issued based on previous queries' results. Thus we model database usage as an *interactive protocol*, a harder setting, where the user sends multiple queries, which are often correlated and may be adaptively malicious (Figure 1).

3 PAC for Databases

In this section, we describe the security model for our privatization layer, PAC-DB, and its corresponding privacy guarantees.

3.1 Security Model

In our security model, we consider two parties: data *contributors* and data *analysts*. Unlike prior work, there is *no* data curator.

Data Contributors. Data contributors are individuals who contribute information to the database. Our study focuses on TPC-H queries which focuses on customers and their purchases. In this setting, our privacy concern is an individual customer; the adversary wants to identify the presence or lack thereof of a customer in the input dataset X, given the query response. We denote this adversarial estimate as $\hat{X}$ and bound the probability that $\hat{X}$ is a successful estimate of X. Our framework, PAC-DB, requires negligible changes to protect other entities (e.g., orders or suppliers).

Data Analysts. Data analysts submit queries to the database. Queries may be malicious and may attempt to extract sensitive data about data contributors based on the query outputs. Regardless of how the queries are constructed, PAC-DB guarantees that the output results provide a bounded posterior advantage for any adversarial inference task. The analyst is *not* required to be familiar with privacy; in particular, the interface for PAC-private queries is indistinguishable to the analyst from the original non-private database. However, privacy-conscious analysts may be able to optimize the privacy-utility tradeoffs.

PAC-DB Usage. PAC-DB is a privatization layer which is compatible with any DBMS. During setup, the privacy concern (e.g., customers) and the per-query privacy budget can be determined once by a trusted party. To privatize a new query, an analyst simply submits the query. **Variance, the analog of DP sensitivity, is automatically calculated by PAC-DB.**

3.2 Privacy Guarantees

3.2.1 PAC-DB Guarantees. Let X denote the sensitive input, which is drawn from a (possibly black-box) distribution D, induced from the underlying database $\mathcal{U}$; let Q denote a query. The distribution D determines the *prior* of the adversary. In our case, for a given dataset of customers $\mathcal{U}$, we generate a distribution D using Poisson

sampling with $p = 0.5$ to generate random subsets of the data [31], enabling a uniform prior of 0.5 on individual membership. Given D, we can formally define the guarantees of a PAC-private query Q.

Definition 2 ($(\delta, \rho, \mathrm{D})$ PAC Privacy [31]). *For a query $Q : \mathcal{X}^* \to O$, some data distribution D, and an inference criterion function $\rho(\cdot,\cdot)$, we say Q satisfies $(\delta, \rho, \mathrm{D})$-PAC Privacy if the following experiment is impossible:*

A user generates data X from distribution D and sends $Q(X)$ to an informed adversary. The adversary who knows D and Q is asked to return an estimation $\hat{X}$ on X such that with probability $\geq (1-\delta)$, $\rho(\hat{X}, X) = 1$. $(1-\delta)$ is referred to as the posterior success rate.

Equivalently, Q can be defined as $(\Delta_f \delta, \rho, \mathrm{D})$ PAC-advantage private if the posterior advantage measured in f-divergence satisfies

$$\Delta_f \delta = \mathcal{D}_f(\mathbf{1}_\delta \| \mathbf{1}_{\delta_o^\rho}) = \delta_o^\rho f(\frac{\delta}{\delta_o^\rho}) + (1-\delta_o^\rho) f(\frac{1-\delta}{1-\delta_o^\rho}),$$

where $(1-\delta_o^\rho)$ represents the optimal prior success rate,

$$\delta_o^\rho = \inf_{X' \in \mathcal{X}^*} \Pr_{X \sim \mathrm{D}}(\rho(X', X) \neq 1),$$

and $\mathbf{1}_\delta$ and $\mathbf{1}_{\delta_o^\rho}$ represent two Bernoulli distributions of parameters δ and δ_o^ρ, respectively. Here, $\mathcal{D}_f$ is some f-divergence.

For this work, we focus on using KL-divergence ($\mathcal{D}_f = \mathcal{D}_{KL}$) where $f(t) = t \log t$. In [31], it is shown that,

$$\mathcal{D}_{KL}(\mathbf{1}_\delta \| \mathbf{1}_{\delta_o^\rho}) = \delta \ln(\frac{\delta}{\delta_o^\rho}) + (1-\delta)\ln(\frac{1-\delta}{1-\delta_o^\rho}) \leq \mathsf{MI}\big(X; Q(X)\big) \tag{1}$$

where $\mathsf{MI}(\cdot,\cdot)$ represents mutual information. Mutual information between two variables X and $Q(X)$ is defined as:

$$\mathsf{MI}(X; Q(X)) = \mathcal{H}(X) - \mathcal{H}(X|Q(X)),$$

where $\mathcal{H}(X)$ is the entropy of X.

Thus, for a given posterior success rate $(1-\delta)$, we can compute the corresponding privacy budget in MI. This allows us to compute Δ, the noise required to privatize $Q(X)$ using Theorem 1 of [27]. When the output, $Q(X) + \Delta$, is released and observed by an adversary, the probability that an adversary can then return an estimate $\hat{X}$ satisfying a given criterion ρ, is at most $(1-\delta)$.

The *secret* that the adversary is trying to discover depends on the half-subset chosen for release — that is, individual membership corresponds to identifying whether a particular datapoint is in the chosen half-subset. The MI guarantee allows us to bound the posterior advantage of *any* task; choices of ρ can be used to represent varying adversarial tasks, including group privacy. However, the *utility* of PAC privacy depends on how representative D is of $\mathcal{U}$, the complete database, for a given query Q. Throughout this work, we provide provable privacy in a black-box manner, *regardless* of the complexity of Q; however, we note that the utility of our privatized query varies based on the *stability* of Q.

3.2.2 Adversarial Attacks. We first define the standard Membership Inference Attack (MIA) [26], formalized to match PAC Privacy.

Definition 3 (Membership Inference Attack [26, 27]). *Given a finite data pool $\mathsf{U} = \{u_1, u_2, \cdots, u_N\}$ and some processing mechanism Q, X is an n-subset of U randomly selected. An informed adversary is asked to return an n-subset $\hat{X}$ as the membership estimation of X after observing $Q(X)$. We say Q is resistant to $(1-\delta_i)$ individual membership inference for the i-th datapoint u_i, if for an arbitrary adversary,*

$$\Pr_{X \leftarrow \mathsf{U}, \hat{X} \leftarrow Q(X)} (\mathbf{1}_{u_i \in X} = \mathbf{1}_{u_i \in \hat{X}}) \leq 1 - \delta_i.$$

Here, $\mathbf{1}_{u_i \in X}$ ($\mathbf{1}_{u_i \in \hat{X}}$) is an indicator which equals 1 if u_i is in X ($\hat{X}$).

Table 2: Mutual information translates to the theoretical maximal posterior success rate for different priors using Equation (1). For a prior of 50%, we can translate mutual information to a DP ϵ value; we use this translation in Figure 5.

Mutual Information	Posterior Success Rate (p_o)		DP ϵ
	Prior p = 50%	Prior p = 25%	Prior p = 50%
1/128	56.241%	30.534%	0.25
1/64	58.815%	32.891%	0.36
1/32	62.434%	36.28%	0.51
1/16	67.490%	41.168%	0.73
1/8	74.464%	48.227%	1.07
1/4	83.789%	58.378%	1.64
1/2	95.181%	72.692%	2.98
1	100%	91.439%	∞
2	100%	100%	∞

With a prior of 50% for an arbitrary datapoint, we can compute the posterior advantage for varying MI, as summarized in Table 2. That is, MI of 1/128 corresponds to a posterior of $\approx$ 56%. Equation (2) of [27] provides the general relationship between MI for PAC and ϵ for DP; we provide the DP ϵ value corresponding to each MI value with a prior of 50% in Table 2, though we note that DP and PAC provide semantically different privacy guarantees.

We also consider a broader set of attacks. For instance, rather than identifying a single record, we can consider the problem of identifying *groups*.

Definition 4 (Group Membership Inference Attack). *Given a finite data pool $\mathsf{U} = \{u_1, u_2, \cdots, u_N\}$ and some processing mechanism Q, X is an n-subset of U randomly selected. For a size-k set of points $T := t_1 \ldots t_k \subset \mathsf{U}$ an informed adversary is asked to return a n-subset $\hat{X}$ as the membership estimation of X after observing $Q(X)$. We say Q is resistant to a $(1-\delta, T)$ generalized membership inference challenge if for an arbitrary adversary, their success rate to return an estimation of all k points correctly is bounded by $(1-\delta)$, i.e.,*

$$\Pr_{X \leftarrow \mathsf{U}, \hat{X} \leftarrow Q(X)} (\sum_{i=1}^{k} \mathbf{1}[\mathbf{1}_{t_i \in X} = \mathbf{1}_{t_i \in \hat{X}}] = k) \leq (1-\delta).$$

For any arbitrary attack, we can first compute the *prior* of the adversary. Assuming a standard prior of 0.5 for individual membership inference, for a specific pair, (x_1, x_2), this prior becomes 0.25. We can then use these priors to compute a bound on the posterior success rate, given a particular privacy budget, as seen in Table 2.

Consider a query with a single output. For a fixed MI budget of 1/128, the posterior success rate for a pair would be $\approx$ 30%, smaller than that for an individual membership inference. In contrast, using naïve composition for DP, the provided ϵ for an individual would increase to 2ϵ for a pair. Tighter composition would require query-specific sensitivity analysis.

3.2.3 Comparison to DP. PAC Privacy provides semantically different guarantees from standard DP. In particular, the PAC Privacy guarantees *depend* on the distribution D of the input data, rather than the input-independent guarantees of DP. Here, we discuss the differing guarantees in the context of a concrete example.

Consider a setting where the complete database $\mathcal{U}$ consists of *all* the census respondents in a country, a classic DP use-case [16] and a differentially-private query Q which asks about the number of residents with salary < \$30,000. In the DP setting, after observing the output of Q, the posterior advantage of identifying whether any datapoint x_0 is in $\mathcal{U}$ is bounded – this is the bounded posterior advantage guarantee for MIA (cf. Definition 3).

In contrast, PAC Privacy provides a guarantee for *the given distribution* D, induced from $\mathcal{U}$. That is, to enable PAC Privacy, we first construct D via Poisson sampling datapoints from $\mathcal{U}$ with $p = 0.5$; concretely, we construct subsets $X_1 \dots X_m$, where each X_i has expected size $|\mathcal{U}|/2$ [27, 31]. D is then the uniform distribution over $X_1 \dots X_m$; when m is sufficiently large, this guarantees that the *prior* that a datapoint x_0 is in X_i is 0.5. Q is then run on a random half-subset X_i drawn from D; the *secret* protected through PAC Privacy is the *choice* of subset. That is, an adversary who knows D, still has a bounded posterior advantage (and success rate) of determining whether x_0 is in X_i — this exactly matches the MIA guarantee discussed in Definition 3, *for the fixed distribution* D. [1]

Consider a setting where every subset returns the same value for a query Q. In this case, PAC Privacy requires zero noise addition for privacy; distinguishing between these subsets is impossible given $Q(X)$. This suggests that any adversarial task on these subsets (e.g., individual or group membership) obtains zero posterior advantage from observing $Q(X)$. In contrast, DP provides an *input-independent* (i.e., dataset-independent) guarantee; in this case, the **global** sensitivity might be higher and often non-zero. Prior works in DP [12, 13] experiment with relaxing the global sensitivity requirement in order to support varying aggregation.

One advantage of the PAC Privacy guarantee is the focus on *arbitrary* adversarial tasks. In particular, classic DP requires composition [8] for group privacy; that is, an $(\epsilon, 0)$ guarantee for an individual becomes $(2\epsilon, 0)$ for a pair. In contrast, an MI guarantee for PAC Privacy applies for arbitrary tasks. In PAC Privacy, the main difference in privacy between the individual and the pair is the *prior*, as defined in Definition 2.

4 PAC Composition

In order to apply PAC to databases, we now prove that the privacy budget composes linearly *even for adaptively generated queries* $Q_1 \dots Q_T$ when the input for each query is sampled *independently* (see Figure 1).

Theorem 1 (PAC Composition). *Denote* $X^* := X_1 \dots X_T$ *as* independent *samples from* D. *Denote* P_x *as the distribution of a random variable* x, O_i *as the output distribution of* Q_i, *and* $\mathsf{P}_{w_i^v}$ *as a distribution over* O_i. *Assume the KL-divergence is bounded as follows*

$$\mathbb{E}_{X_i \sim D}[\mathcal{D}_{KL}(\mathsf{P}_{Q_i(X_i,v)} \| \mathsf{P}_{w_i^v})] \le \bar{r}_i,$$

for $i \in [1, T]$, *any choice of* v *and corresponding* w_i^v. *Denote* $Q^* := (Q_1, \dots Q_T)$ *and* $Q^*(X^*) := Q_1(X_1) \dots Q_T(X_T)$. *There exists a random variable* W *such that:*

$$\mathbb{E}_{X^*}[\mathcal{D}_{KL}(\mathsf{P}_{Q^*(X^*)} \| \mathsf{P}_W)] \le \sum_{i=1}^{T} \bar{r}_i.$$

Proof sketch: Intuitively, we need to represent the *adaptive and adversarial* nature of interactive processes — that is, after observing the *output* of Q_i, the adversary can choose Q_{i+1}. To model this, we provide the adversary with an additional parameter v, which may depend on Q_i. When the inputs are *independently* sampled, the KL-divergence remains bounded by the *sum* of individual divergences.

Proof. First, we consider $T = 2$. To model the adversarial adaptive nature of queries, denote $\mathrm{Adv}(o_1)$ as the adversarial choice after viewing the output of Query 1. Note that there is no such additional input for Query 1.

$$\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_W) = \int \mathsf{P}_{Q^*} \log\left(\frac{\mathsf{P}_{Q^*}}{\mathsf{P}_W}\right)$$

$$= \int_{o_1 \in O_1, o_2 \in O_2} \Pr[Q_1(X_1) = o_1] \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[Q_2(X_2, v) = o_2]$$

$$\log\left[\frac{\Pr[Q_1(X_1) = o_1] \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[Q_2(X_2, v) = o_2]}{\Pr[W = (o_1, o_2)]}\right]$$

We can further break down the denominator:

$$\Pr[W = (o_1, o_2)] = \Pr[W_1 = o_1] \Pr[W_2 = o_2 \mid W_1 = o_1]$$

Since W is arbitrary, we choose $W = (W_1, W_2)$ as follows:

$$\Pr[W_1 = o_1] = \Pr[w_1 = o_1].$$

$$\Pr[W_2 = o_2 \mid W_1 = o_1] = \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[w_2^v = o_2]$$

This implies that:

$$\log\left[\frac{\Pr[Q_1(X_1) = o_1] \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[Q_2(X_2, v) = o_2]}{\Pr[W = (o_1, o_2)]}\right]$$

$$= \log \frac{\Pr[Q_1(X_1) = o_1] \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[Q_2(X_2, v) = o_2]}{\Pr[w_1 = o_1] \sum_{v \in V} \Pr[\mathrm{Adv}(o_1) = v] \Pr[w_2^v = o_2]}$$

We can decompose this using joint convexity:

$$\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_W) = \mathcal{D}_{KL}\left(\sum_v \lambda_v \mathsf{P}_{Q^*,v} \,\middle\|\, \sum_v \lambda_v \mathsf{P}_{W,v}\right)$$

[1] We note that PAC privacy can be meaningfully extended to settings where membership in the database is sensitive by constructing D as a distribution over the sensitive database and a public dataset, with the same distinguishability game described above. As an example, X_i could be a random half-subset of the sensitive database with probability 0.5, or synthetic/public data with probability 0.5. For simplicity, we do not experiment with this setting in this paper.

where $\lambda_v := \Pr[\text{Adv}(o_1) = v]$ and $\mathsf{P}_{Q^*,v}$ is the distribution of P_{Q^*} given $\text{Adv}(o_1) = v$. Thus, by joint convexity,

$$\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_W) \leq \int_{o_1 \in O_1} \Pr[Q_1(X) = o_1] \sum_{v \in V} \Pr[\text{Adv}(o_1) = v] \int_{o_2 \in O_2} \Pr[Q_2(X_2, v) = o_2] \boxed{\cdot \log\left(\frac{\Pr[Q_1(X) = o_1]\Pr[Q_2(X_2, v) = o_2]}{\Pr[w_1 = o_1]\Pr[w_2^v = o_2]}\right)}$$

We first analyze the boxed term:

$$\log\left(\frac{\Pr[Q_1(X_1) = o_1]\Pr[Q_2(X_2, v) = o_2]}{\Pr[w_1 = o_1]\Pr[w_2^v = o_2]}\right) = \log\left(\frac{\Pr[Q_1(X_1) = o_1]}{\Pr[w_1 = o_1]}\right) + \log\left(\frac{\Pr[Q_2(X_2, v) = o_2]}{\Pr[w_2^v = o_2]}\right)$$

Plugging this back in gives us:

$$\begin{aligned}
&\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_W) \\
&\leq \int_{o_1 \in O_1} \left[\Pr[Q_1(X_1) = o_1] \log\left(\frac{\Pr[Q_1(X_1) = o_1]}{\Pr[w_1 = o_1]}\right) \boxed{\sum_{v \in V} \Pr[\text{Adv}(o_1) = v] \int_{o_2 \in O_2} \Pr[Q_2(X_2, v) = o_2]}\right] \\
&+ \int_{o_1 \in O_1} \left[\Pr[Q_1(X_1) = o_1] \sum_{v \in V} \Pr[\text{Adv}(o_1) = v] \right. \\
&\left. \int_{o_2 \in O_2} \Pr[Q_2(X_2, v) = o_2] \log\left(\frac{\Pr[Q_2(X_2, v) = o_2]}{\Pr[w_2^v = o_2]}\right)\right]
\end{aligned}$$

We note that the boxed term is a probability distribution that simply sums to 1. We can then use the fact that

$$\int_{o_1 \in O_1} \Pr[Q_1(X_1) = o_1] \log\left(\frac{\Pr[Q_1(X_1) = o_1]}{\Pr[w_1 = o_1]}\right) = \mathcal{D}_{KL}(\mathsf{P}_{Q_1(X_1)} \| \mathsf{P}_{w_1}).$$

For a fixed v,

$$\int_{o_2 \in O_2} \Pr[Q_2(X_2, v) = o_2] \log\left(\frac{\Pr[Q_2(X_2, v) = o_2]}{\Pr[w_2^v = o_2]}\right) = \mathcal{D}_{KL}(\mathsf{P}_{Q_2(X_2,v)} \| \mathsf{P}^v_{w_2})$$

Thus, our expression becomes:

$$\begin{aligned}
&\mathbb{E}_{X^*}[\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_w)] \leq \mathbb{E}_{X_1 \sim D}[\mathcal{D}_{KL}(\mathsf{P}_{Q_1(X_1)} \| \mathsf{P}_{w_1})] \\
&+ \mathbb{E}_{X_2 \sim D}\Big[\sum_{v \in V} \Pr[\text{Adv}(Q_1) = v] \mathcal{D}_{KL}(\mathsf{P}_{Q_1(X_2,v)} \| \mathsf{P}^v_{w_2})\Big].
\end{aligned}$$

We bound the first term by $\bar{r}_1$. Since the KL-divergence is bounded for *arbitrary* v, the second term is bounded by $\bar{r}_2$. Thus,

$$\mathbb{E}_{X^*}[\mathcal{D}_{KL}(\mathsf{P}_{Q^*} \| \mathsf{P}_W)] \leq \bar{r}_1 + \bar{r}_2$$

The case for general T then follows by induction. □

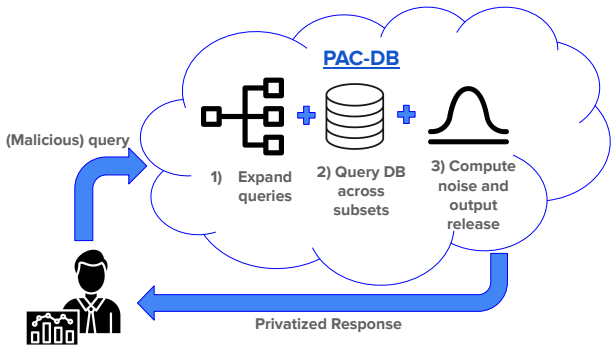


Figure 2: System overview of PAC-DB. We provide a simple three-step template in order to privatize general SQL queries. *No* data curator is required and new queries do not require human input. In Step 1, we *expand* the input query to consider each cell individually. In Step 2, for each cell, we query the database among different randomly-chosen *subsets* of the input dataset. This allows us to compute the stability of the query, and thus, the noise required to privatize it. In Step 3, we then release the noisy output for each cell.

For a given posterior success rate $(1 - \delta)$ and prior $(1 - \delta_o^\rho)$, combining this with classical PAC guarantees allows us to choose the noise required for privatization such that

$$\mathcal{D}_{KL}(\mathbf{1}_\delta \| \mathbf{1}_{\delta_o^\rho}) \geq \sum_{i=1}^{T} \bar{r}_i.$$

That is, we can *bound* the total privacy budget as the sum of MI's on individual queries. This suggests that the key to tight privacy guarantees is *independent random sampling* per query. We discuss the implications of such sampling further in Section 6.3.

5 System Overview

In this section, we provide an overview of the PAC-DB model. We consider a general query Q, mapping from $\mathcal{X}^n \to \mathcal{Y}^{d_1 \times d_2}$. In the most generic setting, we assume that we return d_2 attributes about the input dataset, organized into d_1 groups. We follow a simple, mechanized, three-step process, described in Figure 2.

We first *expand* the response to query Q into individual cells; that is, we consider each output cell Q'_i as an independent (group, attribute) pair. This allows us to re-sample inputs for each cell and apply Theorem 1. For a given cell, we access the database to compute the query response on varying subsets. We compute the variance of the noise required for each cell i as Δ_i. The response for Q'_i then becomes $Q'_i[k] + \mathcal{N}(0, \Delta_i)$, where $Q'_i[k]$ is the response to Q'_i on subset k where k is drawn uniformly at random among all possible subsets. These steps are *agnostic* to the specifics of the query. That is, *any* query can be privatized using the same techniques.

5.1 Step 1: Query Expansion

To expand a query Q, we transform its output from a $d_1 \times d_2$ matrix into individual cells. That is, the d_1 rows correspond to d_1 distinct groups or *index columns*. These index columns are typically publicly

known (e.g., nations); however, for queries where there are *no* index columns, we choose rank as the default index column for consistent processing. In the case where the existence of a non-null value for an index column violates our privacy guarantees, PAC-DB resolves this via null handling (cf. Section 5.5). The response to *each* of the $d_1 d_2$ cells is computed on an *independent* sample of the input data, allowing us to apply Theorem 1.

5.2 Step 2: Query DB across subsets

In order to enable PAC Privacy techniques for databases, we must have a distribution D from which we generate input data.

Constructing the Distribution D. There are several ways to construct the distribution D based on a database $\mathcal{U}$ with varying privacy guarantees. We adapt the structure of [31], where the Poisson distribution is used — that is, to construct a subset X_i, we include each individual $x \in \mathcal{U}$ with probability 0.5. This allows for easy characterization of both *individual* and *group* (e.g., pairs of individual datapoints) privacy. Following [27], for computational tractability, we approximate the Poisson distribution using a finite number of subsets. For our experiments, we choose $m = 128$ subsets; we discuss the tradeoffs for this choice of m in Section 6.3.

Querying the DB. To query the database, we construct a `random_samples` table containing an index column and binary flags for whether each customer is in each subset X_i. For a subset X_i, each row in `customer` is included with probability 50%. Joining `customer` and `random_samples` produces m such groups, corresponding to $\mathcal{S} := X_1, \ldots, X_m$. For a query Q, we run $Q(X_1), \ldots, Q(X_m)$. This provides us with m outputs for each *cell* of the original query output.

5.3 Step 3: Computing Noise and Output Release

Given D and thus, $\mathcal{S}$, we now describe Algorithm 1, which is run on *each cell* of the query. Without loss of generality, let Q' be the first cell. We compute the variance of $Q'(X)$ across the subsets $X_1 \ldots X_m$. We then compute the variance of the *required noise* for a given privacy budget β via Algorithm 2 of [27]; let this be Δ. Let $\mathcal{N}(0, \sigma^2)$ be a random variable drawn from a Gaussian with mean 0 and variance σ^2. When i is drawn uniformly at random from 1 to m, Algorithm 2 of [27] guarantees that

$$\mathrm{MI}(X_i; y_i + \mathcal{N}(0, \Delta)) \leq \beta.$$

We can repeat this process over all cells. Applying Theorem 1 over $d_1 d_2$ cells of the output then guarantees that our overall privacy budget becomes $d_1 d_2 \beta$.

5.4 PAC-DB Example

Consider a simple example query Q, where we would like to count the number of distinct orders and the sum of revenue within two countries A and B, as described in Figure 3. This outputs a matrix, where Q would return the SUM of revenue for A and B and the COUNT of orders for A and B.

We first construct our random samples table by sampling $m = 128$ subsets $X_1 \ldots X_m$. To construct a subset X_i, we use Poisson sampling where each customer x_0 is included in X_i with probability 50%.

Algorithm 1 PAC-DB Algorithm

Input: The input distribution D represented by the set of subsets $\mathcal{S}$, query $Q : \mathcal{X}^n \to \mathcal{Y}$, mutual information requirement β.

(1) $m := |\mathcal{S}|$.
(2) **for** $k = 1, 2, \ldots, m$:
  (a) Compute $y_k = Q(\mathcal{S}[k])$.
(3) Compute the empirical variance σ^2 across the y_k.
(4) Calculate the variance of the required noise as

$$\Delta := \frac{\sigma^2}{2\beta}.$$

(5) Draw i uniformly at random from $1 \ldots m$.
(6) Return $y_i + \mathcal{N}(0, \Delta)$.

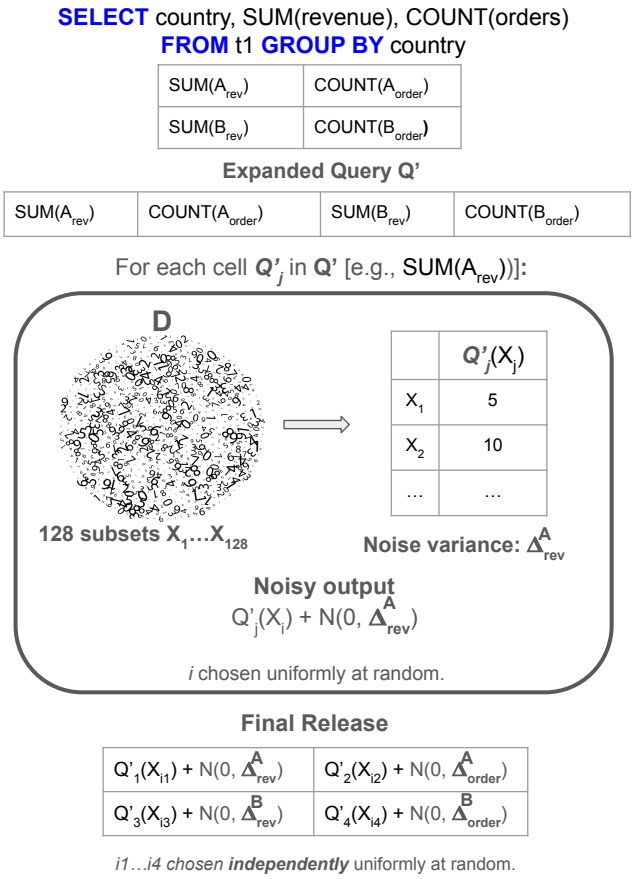


Figure 3: An example workflow of PAC-DB for a query Q, which measures the sum of revenue and the count of orders across two countries A and B. In Step (1), we expand this query into its individual cells. In Step (2), we evaluate each cell across varying subsets of input data (cf. Section 5.2). In Step (3), we use Algorithm 1 to compute the required noise and provide the final privatized release.

In Step 1, we *expand* the query to consider each cell *independently*. We first analyze SUM(A_{rev}) — denote this as Q'_1. In Step 2, to measure the stability of Q'_1, we compute the output distribution

across the subsets; that is, we run $Q_1'(X_i)$ for $i \in [1, m]$. Following Algorithm 1, we compute the variance of the required noise Δ^A_{rev}. Finally, in Step 3, we produce a privatized release. We repeat this process for A_{order}, B_{rev} and B_{order}.

5.5 Additional Cases

So far, our work has only focused on queries with numeric output, where variance can be computed. In order to enable a general privatization protocol with minimal changes required for the analyst, we propose a simple approach for categorical output as well. Consider a query that aims to determine the ID of the seller with the most orders. A classic machine learning technique would represent each category as a one-hot vector (with length corresponding to the *total* number of possible sellers). We then compute the required noise to privatize this vector output using Theorem 1 of [27]. The noisy output vector is then released; the index with the largest value (the argmax) then corresponds to the ID of the top seller.

We further note that releasing the *existence* of index columns with non-null results can sometimes leak information about the database. In prior DP literature [14, 30], τ-thresholding or more complex partition selection [4] techniques are used to address this problem. In our setting, null handling is simply a special case of categorical outputs. That is, we consider null results as simply another unique category and our privacy guarantees follow.

6 Evaluation

6.1 TPC-H Benchmark

The TPC-H benchmark [28] consists of 22 queries issued against a database with 8 tables: `part`, `partsupp`, `lineitem`, `orders`, `region`, `nation`, `supplier` and `customer`. It is a classic decision support benchmark, widely used to evaluate the efficiency and ease-of-use of varying database systems. The benchmark queries are standard business analytics questions regarding, for example, the prices of items in recent sales transactions (Q1). In this work, we consider the table `customer` as our privacy concern and bound the posterior advantage of an adversary identifying the membership of a single (or group) of customers, based on the noisy query outputs. Out of the 22 queries, 3 directly return customer-identifying information and are out of scope for privatization. Prior work in DP has developed tools that can run at most 9 out of the remaining 19 queries without additional analyst labor; we implement *all* 19 queries via PAC-DB with no white-boxing required. Our code and analysis is available at https://github.com/michaelnoguera/pacdb/.

Case 1: Explicit or Implicit Dependence on Customers. Many TPC-H queries explicitly query `customer`; for such a query, we simply join `customer` against `random_samples` such that only the chosen customers remain and re-run the query.

For some of the remaining queries, the original query does not use the customer table, although there exists a functional dependency between tables present in the query and the customer table. Consider Query 4, which estimates the number of orders that were delivered late. Q4 queries `orders` and `lineitem` to identify late deliveries and does not involve the customer table. However, because all orders are placed by a customer, the count of late orders *does* depend on the input set of customers. In the extreme case, if there is only one late order (for Customer X), observing a nonzero entry will immediately identify whether Customer X was in the input dataset. In these cases, making the query compatible with PAC Privacy simply involves adding the necessary intermediate joins to connect `orders` with `customer` to facilitate the join against `random_samples`. This minimal change is shown in Figure 4[2]; it can be easily automated for arbitrary queries.

The majority of the queries (16 out of 22) fall into Case 1.

Case 2: No Dependence on Customers. Certain queries (Q2, Q11, Q16) do not depend on customers at all. For instance, Q2 finds the minimum cost supplier for each part. In this case, revealing this information does not compromise the privacy of any customer; equivalently, $\mathrm{MI}(Q(X), X) = 0$ for Q2, Q11 and Q16. These queries do not need any modification to be privatized via PAC Privacy.

Case 3: Directly Returns Customer Data. As mentioned before, 3 out of 22 queries directly leak customer data: Q3, Q10, Q18. These queries are thus out of scope for privatization.

6.2 Representative Queries

For all numeric queries, we measure utility in terms of relative error. That is, the relative error is computed as:

$$100 * \frac{release - actual}{actual},$$

where *release* represents the noisy output $Q(X_i) + \mathcal{N}(0, \Delta)$ and *actual* is the true output. Results are averaged over 1000 releases.

6.2.1 Query 4: Count. This query focuses on the table `orders`. The goal is to count the number of orders received, where at least one lineitem in the order was received past the commitment date.

```
SELECT
    o_orderpriority,
    count(*) AS order_count
FROM
    orders,                    Subsampling for privatization
    (SELECT * FROM customer
    JOIN random_samples AS rs
    ON rs.row_id = customer.rowid
    AND rs.random_binary = TRUE) AS customer
WHERE
    c_custkey = o_custkey,   ⇐ Added join to customer
    o_orderdate >= …
```

Figure 4: We show a simplified example of Q4. To randomly sample `orders`, we add a join against `customer`, and replace all instances of the original customer table with its subsampled equivalent, obtained by joining `customer` and `random_samples`. This is achieved by adding the SQL shown in the teal box. The joins ensure only orders corresponding to customers chosen in the sample will contribute to the result.

We first consider a DP implementation of this query, since it is within the supported set of aggregations. To enable this, the

[2]A simplified version is presented here; in practice, the random samples table contains m groups corresponding to each of the m subsets.

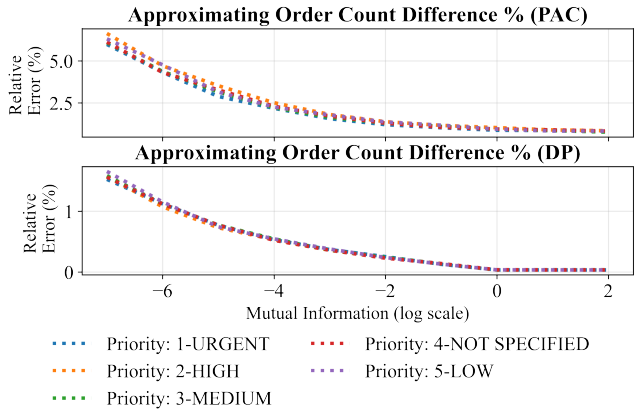


Figure 5: We provide relative errors for Query 4 in the TPC-H framework. *Top:* PAC-DB privatization results for estimating counts for varying orders. The relative error ranges from $< 7\%$ to $< 1\%$. *Bottom:* DP privatization results for the same query. The error ranges between 1.6% to $\approx 0.03\%$.

sensitivity of the query must be computed. When sampling `orders`, the sensitivity is not 1 since there are multiple orders per customer. Thus, the DP implementation must compute the maximum orders per customer. In this dataset, the largest number of orders for a single customer is 41; this is a *distribution-specific* guarantee. If the dataset is updated to add a customer with 50 orders, the DP privatization procedure would need to "clip" the number of records they can contribute to, or update the sensitivity accordingly. We implement the DP version of this query by adding Laplacian noise with scale $41/\epsilon$. This is the best case for a DP implementation where the *exact local sensitivity* is used.

For PAC Privacy, we implement Algorithm 1 with varying values of MI per cell; the PAC Privacy algorithm automatically estimates the noise. We use Table 2 to convert between ϵ and MI for a fixed prior of 0.5. Our results are summarized in Figure 5.

DP provides smaller relative error than the corresponding PAC Privacy guarantees: $\approx 1.7\%$ compared to $\approx 6.6\%$ at small MI. This is due to the fact that this DP bound provides *individual* privacy guarantees with *local* sensitivity. The DP bound provides a tight guarantee for protecting a single user, due to the exact sensitivity bound for the database. If the analyst computing the sensitivity only knows a loose upper bound, the DP error would increase correspondingly (e.g., a bound of 82 would cause the error to double). However, this suggests that when these tight sensitivity bounds are computable *and* constant, DP provides near-optimal error.

In contrast, PAC requires *no* manual analysis on the input data distribution. Further, when considering group privacy, the DP ϵ would increase linearly with the size of the group due to the required composition, but the PAC MI stays the same. That is, to protect the membership of a pair, the required DP error would double, whereas the PAC error would stay the same.

6.2.2 Query 1: Combining Aggregation Functions. In Query 1, we aggregate information about the total business that was billed, shipped, and returned. Query 1 is a particularly complex query with many different aggregation functions in the output:

(1) COUNT: Count of lineitems.
(2) AVG: Average of quantity, price, and discount.
(3) SUM: Sum of quantity and base prices.
(4) Complex aggregations: Sum of the *discounted* prices and the sum of *total* prices.

We provide results for each of these aggregations in Figure 6. We evaluate *each* aggregation over four different groups (corresponding to varying return flags and statuses)

We observe that all queries have reasonable error rates, with below 5% error for MI as low as 1/128. We observe that the group with Return Flag N and Status F has the highest error among all the attributes. This is due to the instability of the query; it has a relatively small number of contributing customers and demonstrates significant variance across subsets. Further, we observe that averages are generally the most stable (with errors $\leq 2.1\%$ for all MI), which aligns with our expectations on the stability of the underlying functionals.

6.2.3 Query 14: Arbitrary Functions. We now consider Query 14 which computes the percentage of revenue due to promotions. This is computed by the following formula:

$$\text{promo-revenue} := 100 \frac{\sum_{i \in promo} cost_i * (1 - discount_i)}{\sum_{i \in all} cost_i * (1 - discount_i)}.$$

That is, promo-revenue represents the percentage of revenue earned from lineitems on promotion as a fraction of all revenue generated. This query shows the difficulty in providing DP sensitivity guarantees for arbitrary functions. Removing a customer can affect *both* the numerator and the denominator. That is, if we consider a case where only one customer bought any item on promotion, then the worst-case sensitivity is proportional to the total cost of their promo items compared to the total cost of all items. Such queries are not supported by prior work [12, 13], and would require significant analyst effort to be supported.

The PAC privatization results for this query are summarized in Figure 7. Although the function is *more* complex than Q4, the relative error is slightly lower: $\approx 5.6\%$ at MI = 1/128. This is a key benefit of PAC Privacy — the error of the query output depends only on the *stability* of the query, rather than its complexity.

6.2.4 Query 20: Arbitrary Functions with Categorical Output. This query tries to determine the set of suppliers who have excess quantity of forest-colored products. The query is particularly interesting, since the set of suppliers is *not* numeric. Since relative error is not computable, we present results on identifying the top supplier accurately. At low MI, we observe that we identify the top supplier correctly $\approx$ 25% of the time; at large MI, our accuracy is above 50%. That is, there are 6 distinct candidates for the top supplier among the 128 subsets. When the privacy budget is tight, we begin to approach a uniformly random choice among the candidates; at larger budgets, we more consistently return the true top supplier.

6.2.5 Remaining Queries. For completeness, we also provide an average relative error for MI = 1/16 for the remaining queries in the TPC-H benchmark. Our results are summarized in Figure 9.

The relative errors of the queries range between $\approx 36\%$ for unstable queries (Q8) to $\approx 1.2\%$ for stable queries (Q6). Q8 measures the market share in a particular within a country for products of

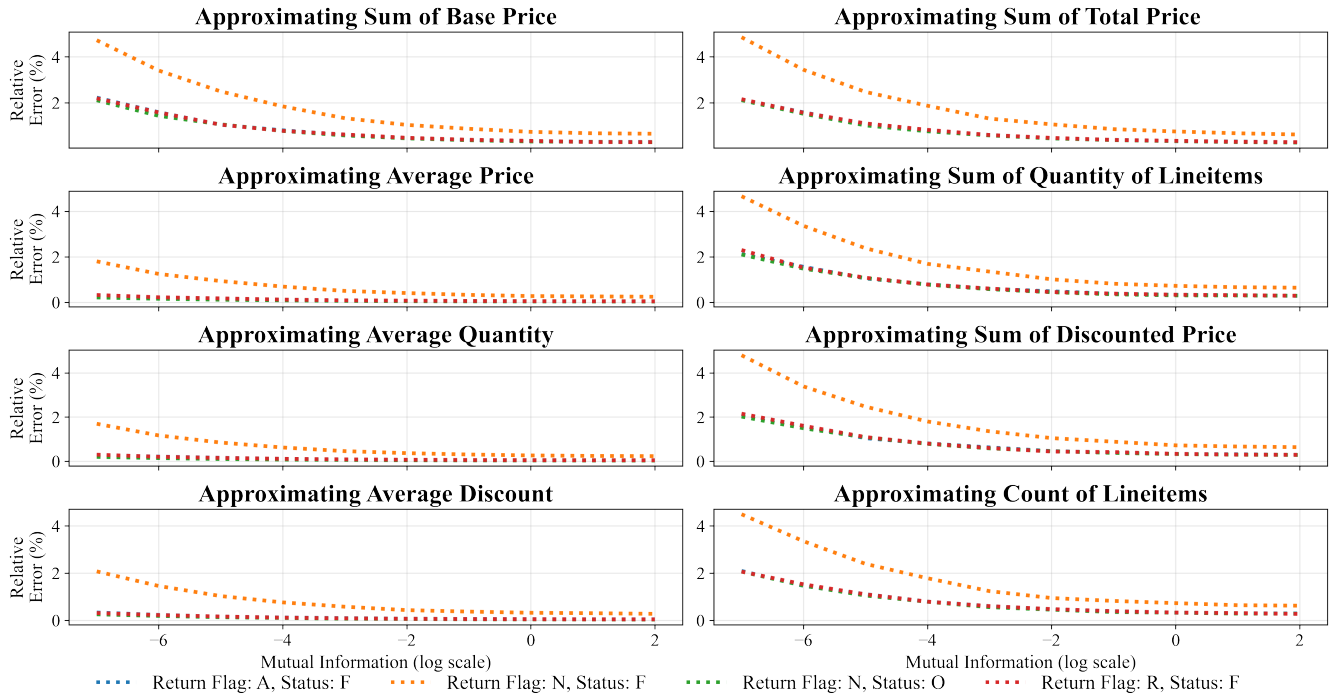


Figure 6: We provide PAC-DB privatization results for *all* groups and *all* aggregation functions in Query 1. The relative errors remain small — below 5% for all groups and aggregations. The "average" queries are the most stable, with errors $\leq 2.1\%$ for all MI. The relative error of each attribute varies with the stability of the query, rather than the complexity.

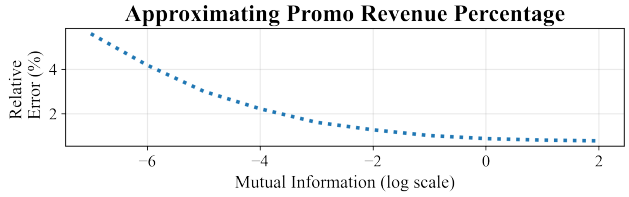


Figure 7: PAC-DB privatization results for the percentage of revenue from promotions (Query 14). The PAC privatization error ranges from $\approx 5.6\%$ for small MI to $\approx 0.8\%$ at large MI.

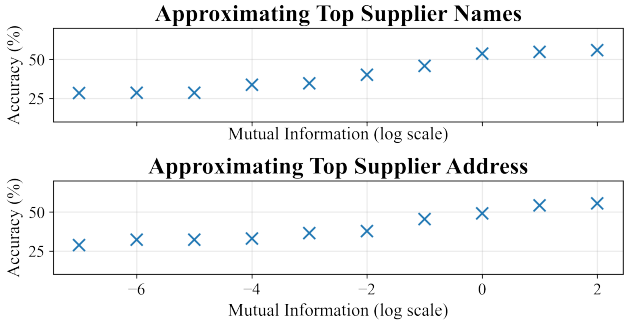


Figure 8: PAC-DB privatization results for the top supplier with an excess of forest-colored parts (Query 20).

a specified type. While this query has high relative error under PAC Privacy, this query would require *significant whiteboxing* to privatize via prior work. In particular, like Q14, Q8 requires tight bounds on the numerator and the denominator and is not supported by prior work. Queries that do not use the customer table (Q2, Q11, Q16) are run unmodified with 0 error.

We further note that PAC-DB implements null handling for Q13, as discussed in Section 5.5. In particular, Q13 creates a histogram of number of customers by the number of orders they have made. While there are many customers who have made 0 orders, there is a negligible number who have made ≥ 38 orders. These groups are sometimes excluded for meaningful privacy guarantees, per our null handling protocol. In particular, PAC-DB returns a response of "null" 2.5% of the time over all the cells in Q13.

6.3 Privacy Guarantees

There are several adversarial tasks that are important to consider. Throughout this section, we use Query 1 as an illustrative example.

6.3.1 *Computing Priors.*

Individual Priors. We first consider a single point x_0 and the following questions:

(1) Was x_0 used in each cell of Q — i.e., did x_0 contribute to the count **and** the sum **and** the average attributes?

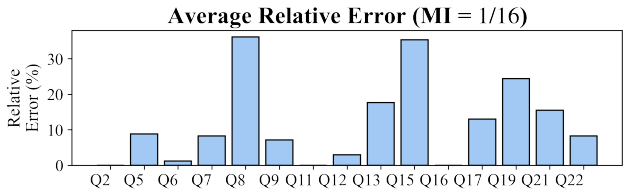


Figure 9: For the remaining queries in the TPC-H framework, we provide average relative errors over all the cells at MI = 1/16. For Q15 and Q21, we measure relative error for the *numeric* cell only.

(2) Was x_0 used in *any* cell of Q — i.e., did x_0 contribute to the count **or** the sum **or** the average attributes?
(3) Was x_0 used in *some specific* cell of Q — i.e., did x_0 contribute to counting the total price of all items sold?

In this work, we are primarily interested in task (3). For completeness, we analyze (1) and (2).

Answering (1) corresponds to identifying exactly which aggregates x_0 contributed to. In each cell, x_0 was included with probability 50%; thus, with 32 outputs, the adversary's probability of correctness is $1/2^{32}$. Answering (2) corresponds to identifying whether x_0 is *ever* used. In each cell, x_0 was included with probability 50%; thus, the probability that x_0 was included in *some* cell is $1 - 1/2^{32}$.

We denote (3) as an *individual* adversarial task. This is the target task for our setting, where we observe a sequence of query responses and quantify overall leakage about an individual datapoint's participation in a *specific cell*. We posit that (3) is particularly appropriate for database settings like the census [16]: that is, identifying whether a given query Q used x_0.

Group Priors. We can ask the same questions about *groups* of datapoints; again, we focus on Task (3). For a fixed group of size k, the prior becomes $1/2^k$, since each datapoint is chosen *independently*.

6.3.2 Computing Posteriors. In each case, the mutual information guarantees compose linearly, as proved in Theorem 1. Consider an MI budget of 1/128. Over the 32 output cells of Query 1, our MI budget would increase to 1/4. This linear composition would continue in the same manner if we considered multiple queries.

Given a *total* MI = 1/4, an individual membership inference task would have a posterior success rate of 84% of identifying whether any individual customer was used in *a particular* cell of the output (Task (3)). If we consider a pair of users, our posterior success rate would be 58%. Of course, adding more noise would reduce MI and the posterior success rate.

6.3.3 Choice of m. As discussed, D is constructed via subsampling; that is, to create a subset X_i, we sample datapoints from $\mathcal{U}$ via Poisson sampling with $p = 0.5$. We construct m such subsets, $X_1 \dots X_m$. In this work, we focus on an adversary who knows the PAC-DB algorithm, D (meaning all the m subsets), and the query Q, but *does not know* which random subset was used in the release. The goal of the adversary is then to return an estimate $\hat{X}$ satisfying the criteria of individual or group membership inference tasks.

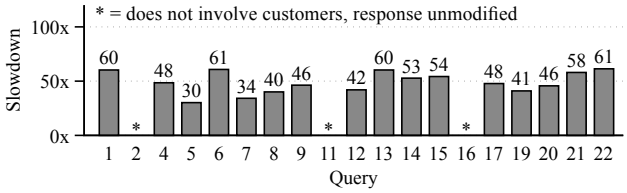


Figure 10: The performance overhead for privatization ranges from 30× to 61× across the TPC-H benchmark. The average performance overhead for modified responses is 49×.

By construction, since $p = 0.5$, an adversary who knows how D is constructed has a prior of 0.5 for individual MIA on x_0 (similarly, 0.25 for a pair). We note that when m is small, these priors may change slightly — that is, in Poisson sampling with $m = 128$, a datapoint may occur in $m' > 64$ subsets due to randomness. As m increases, this change in prior becomes negligible. Further, the probability bound of guessing the chosen, i.e., released subset (as opposed to individual membership) decreases as $1/m$ — we choose $m = 128$ to ensure that this baseline failure probability is below 1%.

While larger m enables lower baseline failure probability and increased stability of our priors, this comes at the cost of performance; we propose that the choice of m should thus be a database system setting. However, we note that increasing m does not significantly impact *utility*. That is, with $m = 1024$, we can measure the change in relative error as:

$$\Delta_{RE} = |RE_{128} - RE_{1024}|.$$

For MI = 1/16, the maximum $\Delta_{RE} = 1.4\%$ (Q17). The change in refusal percentage for Q13 is also stable (0.08%). Thus, we expect minimal changes in our error metrics as m increases beyond 128.

7 Performance

In principle, PAC-DB allows us to trade off computational cost for human labor — rather than manual analysis, the privatization effort is offloaded to simulations. This aligns with the idea that when considering private database analytics, the *privacy budget is typically the limiting factor*, rather than the performance overhead. For instance, prior work with census data [16] shows that the Census Bureau ran only 11 queries per geographic unit for the decennial census redistricting data release.

We provide an end-to-end implementation of PAC-DB with the dual aims of demonstrating PAC's compatibility with database workloads and measuring the computation time overhead of using the PAC privacy technique. Benchmarks used a Dell PowerEdge R6615 server with an AMD EPYC 9354P CPU and 188 GiB of RAM, provisioned on CloudLab [6]. Datasets were generated using the DuckDB TPC-H extension and stored in a single-file DuckDB database on a local NVMe SSD.

In Figure 10, we compare the runtime of private queries in PAC-DB against non-private queries in DuckDB for all TPC-H queries at scale factor 1. The results are averaged over three trials. Private queries ran 30× to 61× slower than their non-private equivalents. Unmodified queries (Q2, Q11, Q16) incur no performance cost.

To further analyze runtime, we timed three parts of each query execution: *DuckDB Execution*, which constructs the random samples

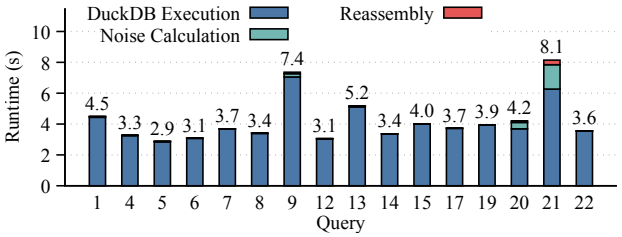


Figure 11: A step-by-step breakdown shows that most of the execution time is spent in DuckDB Execution. Queries with larger output tables (e.g., Q9, Q20, Q21) show more time spent in Noise Calculation.

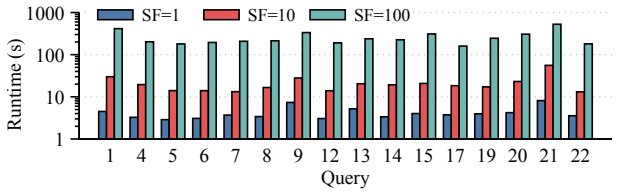


Figure 12: We measure PAC-DB performance on SF= 1, 10, 100 for the TPC-H benchmark.

table and computes the query response over each sample; *Noise Calculation*, which runs Algorithm 1 to calculate the noise and privatize the output; and *Reassembly*, which updates the output table with these private values. Results are shown in Figure 11.

DuckDB Execution is the largest component of runtime. It executes the query as a prepared statement in DuckDB once per sample, each time joining the customer table against a different random sample set to use only the chosen customer rows. Where necessary, we optimized queries (e.g. Q17, Q21) using standard techniques like pushing down static filters and materializing the results for reuse across all $m = 128$ samples. However, many opportunities for further optimization remain. Time spent on *Noise Calculation* scales with the size of the output table, and is only noticeable for queries with larger output tables (e.g. Q9, Q20, Q21). Likewise, *Reassembly* consolidates the noisy outputs into a table, which for small tables takes minimal time. We note that our simple implementation writes intermediate states to disk between steps, incurring unnecessary serialization and disk I/O overhead. For production uses, *Noise Calculation* and *Reassembly* could be added as a native database extension to avoid this expensive data movement.

We also analyze performance on larger databases using TPC-H with scale factors (SF) 1, 10, and 100. The results, presented in Figure 12, show that runtimes scale reasonably with data volume.

8 Related Work

Local DP and Shuffle DP. Local differential privacy relaxes the trust requirements on the data curator. In particular, data contributors add noise to their data *before* submitting it to the data curator (e.g., [29]). Thus, the curator holds only differentially-private data and does *not* need to be trusted. Unfortunately, the local model requires a lot of noise compared to the central model in [12, 13] and is therefore not usable in most settings [22].

Similarly, in shuffle DP [2], data contributors add noise to their local data and submit it to a trusted shuffler, which anonymizes and shuffles the data before sending it to the curator. Shuffling results in privacy amplification [35]; however, the required noise is sensitive to churn among honest users and its amplification remains open for more general data processing other than linear aggregation.

Synopses for Provable Privacy. An alternate approach to providing differential privacy is providing private *views* [11]. That is, rather than providing privacy at the query level, [15] suggests instead providing a synopsis of the data. This synopsis will create a differentially-private summary of the original dataset, with a fixed privacy budget. This data can then be used for an unlimited number of arbitrarily complex queries, with no further privacy degradation.

However, there are two main drawbacks with this approach. First, simple summaries only work for linear queries; prior work [15] suggests that these will not work for complex joins. For meaningful utility guarantees for arbitrary relations, we must allow *multiple* views. In order to construct these views, we must have access to a representative workload — that is, we need to know *a priori* the set of queries that will be requested. Second, each differentially-private view will have a separate sensitivity calculation, which may be arbitrarily complicated and require analyst effort to enable.

Privacy through Cryptographic Techniques. These techniques focus on a fundamentally *different* threat model. These works assume that the analyst is allowed to have access to the raw query output; the server or individual data contributors are untrusted, and data of a particular contributor needs to be protected from the others.

A simple instantiation of this framework is to simply execute the SQL queries over *encrypted* data, as proposed in [23]. This typically requires heavy, SQL-aware encryption schemes and are hard to implement and maintain. In contrast, [10] encrypts the database and every query, trading off cryptographic requirements for secure hardware. In a slightly different problem setting, [1] considers *data sharing* in the federated setting, where a trusted broker uses secure multiparty computation to return a complete response.

To provide data privacy against the *analysts*, these techniques are often layered with DP [22]. For instance, [21] creates a new query language in order to protect the data contributors from malicious data curators *and* analysts. However, this work only extends to count queries, due to the complexity of query rewriting required for both DP *and* the cryptographic primitives used. PAC-DB allows us to simplify and automate the work required to provide privacy for data contributors from analysts; layering PAC-DB with cryptographic techniques is an interesting area for future work.

9 Conclusions

In this work, we showed how PAC Privacy can be used to privatize general SQL queries. We expand upon the PAC Privacy framework to provide strong theoretical guarantees for adaptive, adversarial composition, a *crucial* requirement to provide a usable private database. We prove that independent re-sampling allows for tight privacy guarantees and use this to create a simple 3-step template for the privatization of general SQL queries, via PAC Privacy. Our template is compatible with arbitrary database management systems with no trusted curator required.

To the best of our knowledge, PAC-DB is the first system to privatize general SQL queries without additional analyst input; using a DuckDB-based implementation we provide results on 19 out of the 22 TPC-H benchmark queries (all queries that do not directly leak customer information). This is a significant expansion in the *capability* of private databases; rather than requiring query white-boxing or manual data curation, we can *automatically* privatize queries in a black-box manner. An exciting aspect of future work is to create more efficient PAC privatization procedures, both in terms of computational speedup and noise reduction.

References

[1] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel Kho, and Jennie Rogers. 2017. SMCQL: secure querying for federated databases. *Proc. VLDB Endow.* 10, 6 (Feb. 2017), 673–684. https://doi.org/10.14778/3055330.3055334

[2] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Marc Rudominer, Ushasree Kode, Julien Tinnés, and Bernhard Seefeld. 2017. Prochlo: Strong Privacy for Analytics in the Crowd. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. ACM, Shanghai, China, 441–459. https://doi.org/10.1145/3132747.3132769

[3] Surajit Chaudhuri and Umeshwar Dayal. 1997. An overview of data warehousing and OLAP technology. *SIGMOD Rec.* 26, 1 (March 1997), 65–74. https://doi.org/10.1145/248603.248616

[4] Damien Desfontaines, James Voss, Bryant Gipson, and Chinmoy Mandayam. 2022. Differentially private partition selection. *Proceedings on Privacy Enhancing Technologies* 2022, 1 (2022), 339–362. arXiv preprint arXiv:2006.03684.

[5] Wei Dong, Zijun Chen, Qiyao Luo, Elaine Shi, and Ke Yi. 2024. Continual Observation of Joins under Differential Privacy. *Proc. ACM Manag. Data* 2, 3, Article 128 (May 2024), 27 pages. https://doi.org/10.1145/3654931

[6] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. 2019. The Design and Operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. 1–14. https://www.flux.utah.edu/paper/duplyakin-atc19

[7] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.

[8] Cynthia Dwork and Aaron Roth. 2014. The Algorithmic Foundations of Differential Privacy. *Found. Trends Theor. Comput. Sci.* 9, 3–4 (aug 2014), 211–407. https://doi.org/10.1561/0400000042

[9] Juanru Fang, Wei Dong, and Ke Yi. 2022. Shifted Inverse: A General Mechanism for Monotonic Functions under User Differential Privacy. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)* (Los Angeles, CA, USA). Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3548606.3560567

[10] Alexey Gribov, Dhinakaran Vinayagamurthy, and Sergey Gorbunov. 2017. StealthDB: a Scalable Encrypted Database with Full SQL Query Support. *Proceedings on Privacy Enhancing Technologies* 2019 (11 2017). https://doi.org/10.2478/popets-2019-0052

[11] Alon Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10, 4 (Dec. 2001), 270–294. https://doi.org/10.1007/s007780100054

[12] Noah Johnson, Joseph P. Near, Joseph M. Hellerstein, and Dawn Song. 2020. Chorus: a Programming Framework for Building Scalable Differential Privacy Mechanisms. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. 535–551. https://doi.org/10.1109/EuroSP48549.2020.00041

[13] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proc. VLDB Endow.* 11, 5 (Oct. 2018), 526–539. https://doi.org/10.1145/3177732.3177733

[14] Aleksandra Korolova, Krishnaram Kenthapadi, Nina Mishra, and Alexandros Ntoulas. 2009. Releasing search queries and clicks privately. In *Proceedings of the 18th International Conference on World Wide Web* (Madrid, Spain) *(WWW '09)*. Association for Computing Machinery, New York, NY, USA, 171–180. https://doi.org/10.1145/1526709.1526733

[15] Ios Kotsogiannis, Yuchao Tao, Xi He, Maryam Fanaeepour, Ashwin Machanavajjhala, Michael Hay, and Gerome Miklau. 2019. PrivateSQL: a differentially private SQL query engine. *Proc. VLDB Endow.* 12, 11 (July 2019), 1371–1384. https://doi.org/10.14778/3342263.3342274

[16] Cory McCartan, Tyler Simko, and Kosuke Imai. 2023. Making Differential Privacy Work for Census Data Users. *Harvard Data Science Review* 5 (05 2023). https://doi.org/10.1162/99608f92.c3c87223

[17] Frank D. McSherry. 2009. Privacy integrated queries: an extensible platform for privacy-preserving data analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data* (Providence, Rhode Island, USA) *(SIGMOD '09)*. Association for Computing Machinery, New York, NY, USA, 19–30. https://doi.org/10.1145/1559845.1559850

[18] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2011. Dremel: interactive analysis of web-scale datasets. *Commun. ACM* 54, 6 (June 2011), 114–123. https://doi.org/10.1145/1953122.1953148

[19] Noman Mohammed, Samira Barouti, Dima Alhadidi, and Rui Chen. 2015. Secure and Private Management of Healthcare Databases for Data Mining. In *2015 IEEE 28th International Symposium on Computer-Based Medical Systems*. 191–196. https://doi.org/10.1109/CBMS.2015.54

[20] Prashanth Mohan, Abhradeep Thakurta, Elaine Shi, Dawn Song, and David E. Culler. 2012. GUPT: privacy preserving data analysis made easy. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*, K. Selçuk Candan, Yi Chen, Richard T. Snodgrass, Luis Gravano, and Ariel Fuxman (Eds.). ACM, 349–360. https://doi.org/10.1145/2213836.2213876

[21] Arjun Narayan and Andreas Haeberlen. 2012. DJoin: differentially private join queries over distributed databases. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation* (Hollywood, CA, USA) *(OSDI'12)*. USENIX Association, USA, 149–162.

[22] Joseph P. Near and Xi He. 2021. Differential Privacy for Databases. *Foundations and Trends in Databases* 11, 2 (2021), 109–225. https://doi.org/10.1561/1900000066

[23] Raluca Ada Popa, Catherine M. S. Redfield, Nickolai Zeldovich, and Hari Balakrishnan. 2011. CryptDB: protecting confidentiality with encrypted query processing. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (Cascais, Portugal) *(SOSP '11)*. Association for Computing Machinery, New York, NY, USA, 85–100. https://doi.org/10.1145/2043556.2043566

[24] Davide Proserpio, Sharon Goldberg, and Frank McSherry. 2014. Calibrating data to sensitivity in private data analysis: a platform for differentially-private analysis of weighted datasets. *Proc. VLDB Endow.* 7, 8 (April 2014), 637–648. https://doi.org/10.14778/2732296.2732300

[25] Tajinder Saini and Sachin Lalar. 2024. *Unveiling Privacy, Security, and Ethical Concerns of ChatGPT*. (pp. 202–215). https://doi.org/10.4018/979-8-3693-6824-4.ch011

[26] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 3–18.

[27] Mayuri Sridhar, Hanshen Xiao, and Srinivas Devadas. 2025. PAC-Private Algorithms . In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3839–3857. https://doi.org/10.1109/SP61157.2025.00034

[28] Transaction Processing Performance Council. 2023. *TPC-H Benchmark Specification*. Technical Report. Transaction Processing Performance Council. http://www.tpc.org/tpch/ Revision 3.0.1.

[29] Leixia Wang, Qingqing Ye, Haibo Hu, and Xiaofeng Meng. 2024. PriPL-Tree: Accurate Range Query for Arbitrary Distribution under Local Differential Privacy. *Proc. VLDB Endow.* 17, 11 (July 2024), 3031–3044. https://doi.org/10.14778/3681954.3681981

[30] Royce Wilson, Celia Zhang, William Lam, Damien Desfontaines, Daniel Simmons-Marengo, and Bryant Gipson. 2020. Differentially Private SQL with Bounded User Contribution. *Proceedings on Privacy Enhancing Technologies* 2020 (04 2020), 230–250. https://doi.org/10.2478/popets-2020-0025

[31] Hanshen Xiao and Srinivas Devadas. 2023. PAC Privacy: Automatic Privacy Measurement And Control Of Data Processing. In *Advances in Cryptology – CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part II* (Santa Barbara, CA, USA). Springer-Verlag, Berlin, Heidelberg, 611–644. https://doi.org/10.1007/978-3-031-38545-2_20

[32] Xiaokui Xiao and Yufei Tao. 2008. Output perturbation with query relaxation. *Proceedings of the VLDB Endowment* 1, 1 (2008), 857–869.

[33] Bangzhou Xin, Wei Yang, Yangyang Geng, Sheng Chen, Shaowei Wang, and Liusheng Huang. 2020. Private FL-GAN: Differential Privacy Synthetic Data Generation Based on Federated Learning. In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2927–2931. https://doi.org/10.1109/ICASSP40776.2020.9054559

[34] Dan Zhang, Ryan McKenna, Ios Kotsogiannis, George Bissias, Michael Hay, Ashwin Machanavajjhala, and Gerome Miklau. 2020. ϵKTELO: A Framework for Defining Differentially Private Computations. *ACM Trans. Database Syst.* 45, 1, Article 2 (Feb. 2020), 44 pages. https://doi.org/10.1145/3362032

[35] Úlfar Erlingsson, Vitaly Feldman, Ilya Mironov, Ananth Raghunathan, Kunal Talwar, and Abhradeep Thakurta. 2019. Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity. In *Proceedings of the 2019 ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2468–2479. https://doi.org/10.1137/1.9781611975482.150